

Sixdof Falcon with Mavsdk Examples

Falcon

The Sixdof Falcon development kit provides a programmatic way of interacting with the Sixdof sensor. Before moving onto the provided examples please see the documentation for Falcon. It is necessary to first understand the various tracking modes provided, before moving onto a specific example.

These examples are based on two of the tracking modes. In Sixdof tracking mode, a precise map of three or more beacons must be specified. In this mode the full 6dof (position and orientation) or the sensor is reported. This mode is typically used for precise drone guidance where orientation is required.

In Relative Beacon tracking mode, the sensor reports beacon locations relative to its own coordinate system. In this mode a map is not required, but no orientation information is received. Typically, this mode is used in tandem with a high-powered LED in order to find a landing target from high elevations.

A drone guidance system can take advantage of both modes, however, only one of these modes can be running at a time. Additionally, there are other modes available in the SDK, mainly the Relative Angles mode, which is meant for identifying the angle to a high-powered LED from very high elevations.

Mavsdk

In these examples the C++ distribution of Mavsdk was used to interact with a drone. Mavsdk enables communications using the Mavlink protocol, which can be used to send commands to any drone that implements the Mavlink protocol, this includes both PX4 and Ardupilot drones.

Examples

Sixdof Hover and Landing

In this example the Sixdof tracking mode is used for an assisted hover and precise landing. The example can be found in the folder “DroneSixdofHover”.

Relative Beacon Hover and Landing

In this example the Relative Beacon tracking mode is used for an assisted hover. The example can be found in the folder “RelLightHover”.

Mixed Landing

In this example, first the Relative Beacon tracking mode is used to bring the drone from a high elevation down to within a few meters of the landing target, then the mode is switch to Sixdof tracking for a precision landing. The example can be found in the folder “TakeoffAndLandDemo”.

Core Concepts

Offboard mode

Offboard mode is the API feature that is used to control the drone in these examples. It allows us to control the drone in two ways that are relevant to us. The first is with body velocity, this enables us to specify the desired velocity of the drone in its own coordinate system. This is mostly used for setting the velocity to 0 in order to get the drone to stay still for short periods of time. The more

useful API feature of Offboard mode is the ability to set the desired position of the drone in NEDY coordinates. Where NEDY stands for North East Down Yaw. NEDY is a local coordinate system that is aligned with compass North. The tricky part of using NEDY coordinates is that the origin of the coordinate system is mostly arbitrary, and is set when the drone is turned on. To correct for the unknown origin we have to constantly query the drones own estimate of its NEDY coordinate. This will be discussed further in the next section.

Other Mavsdk features can be used to control the drone such as Mocap, however they were not included in these examples.

Coordinate Systems

One of the difficulties in drone guidance systems is keeping track of all the coordinate systems involved. Going forward I will use the following notation:

$$P_A^B = P_{to}^{from}$$

Where P refers to a 3D transformation matrix of A with respect to B. Another way of thinking about it is the transformation from B to A. Typically this transformation is represented with a 4x4 transformation matrix. One very useful property of transformation matrices is that their inverse represents the inverse transformation:

$$P_B^A = inv(P_A^B)$$

The output data of the Sixof tracking mode is the position of the sensor with respect to the landing target P_{Sensor}^{Target}. In order to control the drone with Offboard mode we need to calculate the desired position of the drone relative to NEDY coordinate system, P_{Target}^{NED}. Which can be done by the following equation:

$$P_{Target}^{NED} = P_{Drone}^{NED} * P_{Sensor}^{Drone} * P_{Target}^{Sensor}$$

Where $P_{Target}^{Sensor} = inv(P_{Sensor}^{Target})$, P_{Sensor}^{Drone} is determined by where the sensor is placed on the drone, and P_{Drone}^{NED} can be queried from Mavsdk at runtime. This logic is implemented in the function “sixdof_wrt_map_to_nedy”, and similar logic is implemented to convert Relative Beacon positions to NEDY coordinates as well, see the “relight_wrt_map_to_ned” function.